

ALGORIK

GCP Full Platform Architecture Blueprint

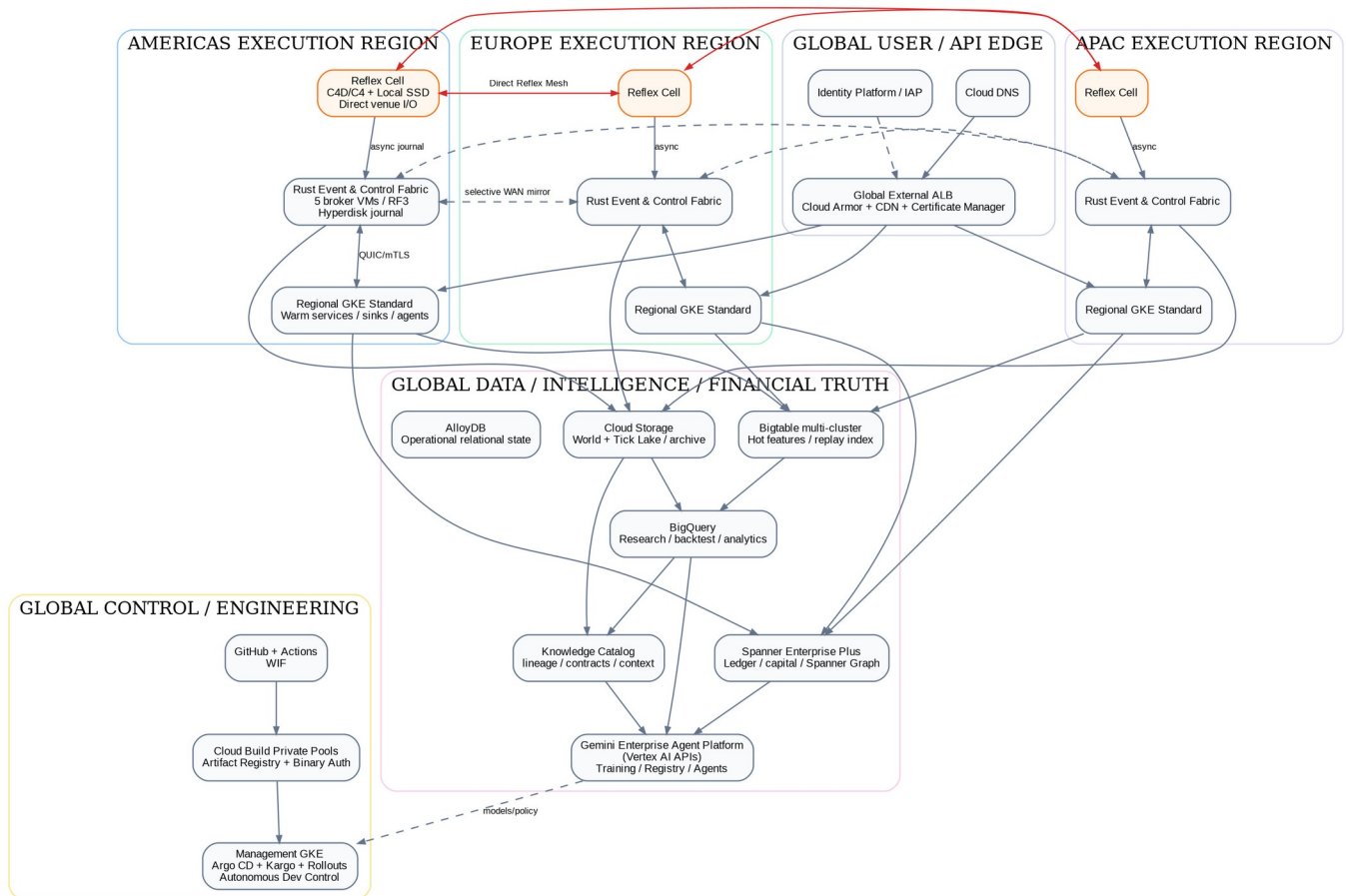
Native Rust Event & Control Fabric · speed-first execution · global cognition · resilient financial state · GitOps · autonomous development

Version 2.1 — September 2026

Companion cloud/platform blueprint for Algorik Financial AGI v11.5

Architecture thesis

Three paths, three jobs: direct Venue/Reflex I/O for the hot path; direct Reflex Mesh for latency-sensitive multi-region coordination; and the Algorik Rust Event & Control Fabric for durable asynchronous control, journals, outcomes, telemetry and replay. Managed GCP services scale the system around those paths without becoming synchronous dependencies of execution.



Every component below exists for a specific latency, durability, availability, security, or operability reason.

1. Executive Architecture

Algorik is not deployed as one application or one cluster. It is a set of deliberately separated planes with different latency and consistency requirements. The fastest path is regional and deterministic; durable messaging is a native Rust subsystem; financial truth is strongly consistent; world/tick data is replayable; cognition is elastic and asynchronous; and every production change is reproducible from Git. External world information is treated as pass-through sensory input: the platform re-fetches sources when needed and persists only compact derived knowledge plus provenance references.

Plane	Primary runtime	Why
Reflex Execution	Dedicated Compute Engine C4D/C4-class VMs	Lowest jitter and direct venue sessions; local books/features/models/risk; no service-mesh or database dependency.
Reflex Mesh	Direct peer QUIC between regional Reflex Cells	Fast bounded coordination for 2–20-leg opportunities; ephemeral by design.
Rust Event & Control Fabric	Dedicated Rust broker VMs per region	One native contract for durable pub/sub, journals, replay, backpressure, control and outcomes; replaces core Pub/Sub/Kafka.
Regional Service Plane	Regional GKE Standard	Elastic warm services: asset/risk/capital, evidence, replay, fabric sinks, APIs, package distribution.
Data / Intelligence	GCS, Bigtable, BigQuery, Knowledge Catalog, Spanner Graph, Gemini Enterprise Agent Platform / Vertex APIs	Pass-through world sensing plus retained tick/internal history, compact provenance/knowledge, research, world models, training, agents and symbolic/quantum work.
Financial Truth	Spanner Enterprise Plus	Strongly consistent ledger, positions, obligations, capital grants and idempotent outcome processing.
Engineering / Control	GitHub, Cloud Build private pools, Artifact Registry, Config Sync, Argo CD/Kargo/Rollouts	Immutable supply chain, GitOps promotion, progressive delivery and autonomous development through the same controls.

Explicitly removed from the core runtime
Google Pub/Sub and Managed Kafka are not internal Algorik runtime dependencies. They are not needed for service-to-service control, tick journaling, outcomes or replay. If a partner requires Kafka/Pub/Sub compatibility, a stateless bridge may exist at an external boundary, but Algorik semantics remain defined by the Rust fabric.

2. Non-Negotiable Architecture Rules

2.1 World Data Pass-Through / Knowledge Persistence Invariant

External world data is a sensory stream, not a storage domain. Crawlers, APIs and feeds deliver content into transient processing workers. The payload is parsed, evidence-scored and distilled into durable knowledge; the source payload is then discarded. GCP storage is used for the knowledge that Algorik creates, not for mirroring the public world.

- Transient fetch: crawler/scout pods or Cloud Run/GKE workers stream source bytes through memory or encrypted ephemeral disk only; no default GCS landing bucket for external-world content.
- Source Manifest Registry: Spanner/AlloyDB stores canonical URI/provider ID, retrieval time, version/etag, hash, entitlement, cursor, freshness, parser lineage and re-fetch instructions.
- Knowledge Graph: Spanner Graph stores entities, relationships, temporal facts, causal edges, source references, beliefs, confidence and contradictions - never copied documents.
- Fast Knowledge State: Bigtable stores high-rate derived features, entity/event state, ambient-model state and time-indexed knowledge deltas.

- Cheap Knowledge Packs: compact protobuf/Parquet snapshots, embeddings, summaries, world-model deltas and model artifacts may be stored in GCS with lifecycle tiers because they are Algorik-derived and dramatically smaller than source corpora.
- Historical world analysis: re-fetch from the authoritative source/provider using the manifest, then process transiently again. If licensed providers expose historical APIs, query them on demand rather than copying their whole corpus.
- Tick/order-book and Algorik-owned financial/execution data are distinct retention classes and can be archived when rights/policy require replay, accounting, audit or model training.

#	Rule	Consequence
1	Hot path owns its dependencies	Between tick and order: only local process memory, local deterministic risk, local session state and the venue adapter. No GKE, Spanner, Bigtable, Redis, AI endpoint or fabric acknowledgment.
2	Async does not mean unimportant	Control, outcomes, journals and telemetry are durable where required, but they execute after or beside the reflex decision.
3	One internal event model	Every asynchronous event carries event_id, producer_epoch, schema_id/version, region, logical timestamp, source timestamp, trace_id, ordering key, integrity checksum and provenance metadata.
4	At-least-once + idempotency	Do not claim magical global exactly-once. Consumers use event IDs, partition sequence and fencing epochs; financial writers enforce uniqueness transactionally.
5	Regions fail independently	Loss of global cognition or another region cannot stop a healthy Reflex Cell. Each dependency has a degradation envelope.
6	Durability is per topic	Control and outcomes get quorum durability; telemetry can be sampled/dropped; research streams optimize throughput.
7	State ownership is explicit	Spanner owns financial truth; Fabric owns event history/offsets; GCS owns retained tick/internal archives and compact derived knowledge artifacts; external world payloads are transient; Bigtable owns hot derived features/state; caches own nothing authoritative.
8	Git and signed digests are deployment truth	No mutable tags, console-only changes, hand-copied models, or secret values in Git.
9	Autonomous agents obey the same pipeline	Agents can create PRs, run experiments and diagnose incidents, but production change requires tests, attestations, promotion policy and rollback.
10	Cloud latency has physics	GCP regional execution is fast cloud trading, not exchange colocation. Venue-specific benchmarks determine region and topology; true sub-millisecond colocation would require edge/colo nodes outside ordinary cloud regions.

3. Global Region and Failure-Domain Strategy

Start with three execution regions - Americas, Europe and APAC - but treat exact GCP regions as a measurement outcome, not a hard-coded architectural preference. Select them by venue RTT, market-data RTT, legal residency, available machine series, Interconnect options, and proximity to counterparties. The examples in diagrams are logical regions.

- **Per execution region:** separate Reflex VPC, Fabric VPC, Service VPC/GKE cluster, regional landing storage, regional observability collectors and controlled egress.

- **Global/shared:** management/build plane, multi-region ledger, global research data, model registry, Knowledge Catalog and identity/security services.
- **Zonal failure:** Reflex standby, Fabric RF3 partition replicas across zones, GKE regional control plane and multi-zone node pools.
- **Regional failure:** healthy regions continue; critical Fabric topics have selective cross-region mirrors; ledger/data services remain available according to managed-service topology.
- **Capacity:** reserve hot Reflex capacity with reservations/commitments; autoscale warm/data/AI components where scaling time is compatible with workload behavior.

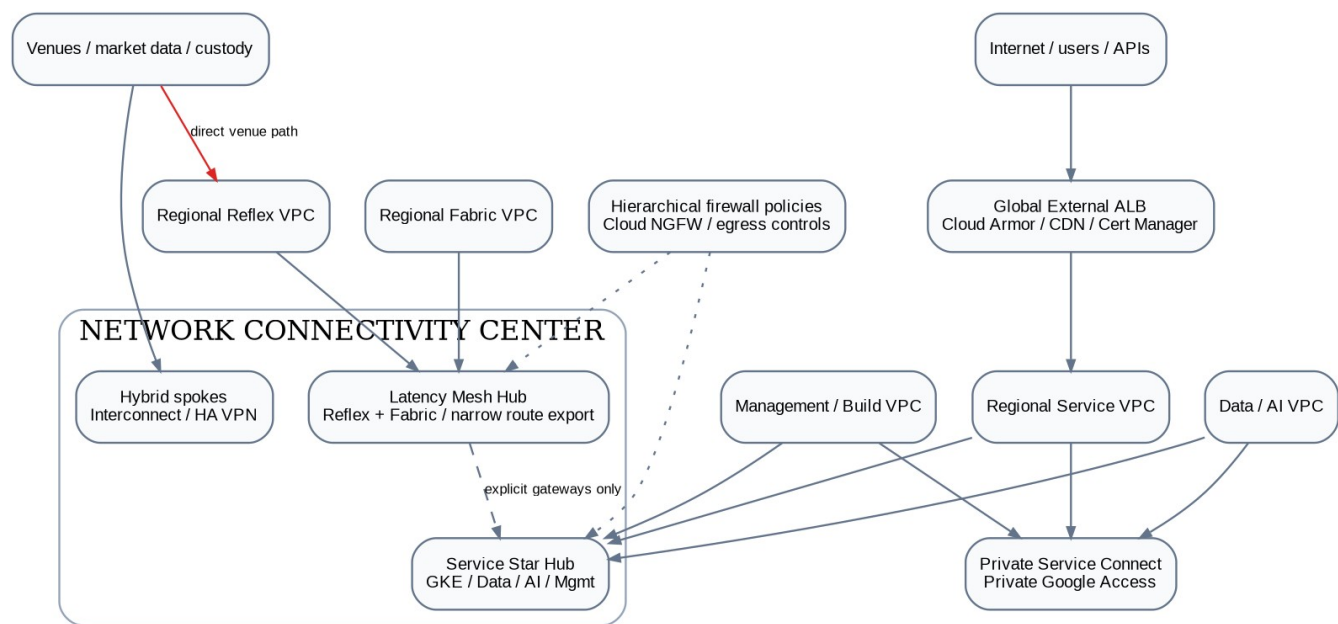
4. GCP Organization, Folder and Project Model

Folder / Project pattern	Resources	Reason
org/platform-foundation	Org Policy, IAM/PAM patterns, audit sinks, SCC, billing exports	Central guardrails independent of applications.
prod/network-core	NCC hubs, shared DNS, hybrid connectivity, firewall policy, PSC endpoints	Network authority separated from workloads.
prod/exec-<region>	Reflex VPC + C4D/C4 Reflex VMs	Smallest blast radius for latency-critical execution.
prod/fabric-<region>	Fabric VPC + broker VMs + journal disks + archiver identities	Messaging failure cannot directly reconfigure trading compute.
prod/services-<region>	Regional GKE Standard, Redis, regional APIs/sinks	Warm services scale independently of Reflex and Fabric.
prod/ledger	Spanner ledger/accounting/capital databases, KMS keys	Financial truth isolated from noisy analytics.
prod/world-model	Spanner Graph / world-model stores	Graph/cognition query load isolated from ledger.
prod/data	GCS retained tick/internal archive + compact knowledge packs, Bigtable, BigQuery, Knowledge Catalog	Data governance and analytics boundary.
prod/ai	Agent Platform/Vertex training, registry, agent runtime, GPU/TPU jobs	AI permissions and quota separated from financial state.
prod/engineering	Cloud Build private pools, Artifact Registry, signing/attestation, GitOps management cluster	Software supply-chain boundary.
nonprod/*	Mirrors same logical boundaries at lower scale	Architecture parity without production access.

Why not one giant Shared VPC?

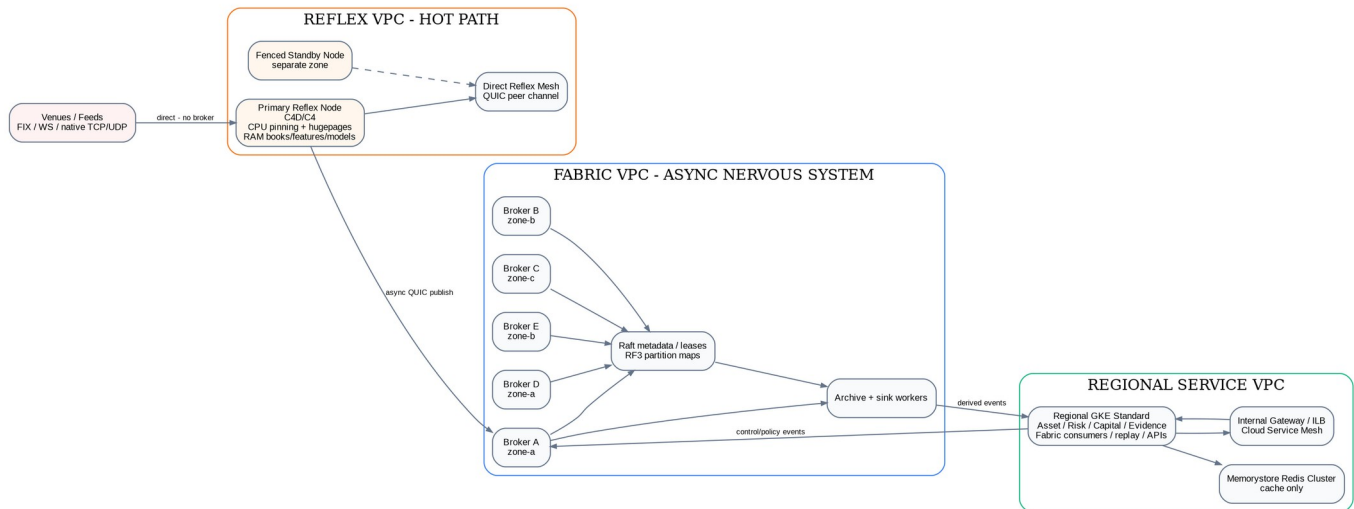
A single Shared VPC is easy initially but broadens routing and change blast radius. v2 uses NCC to connect intentionally separated VPCs. Shared VPC may still be used inside a domain/folder where centralized subnet ownership is useful, but Reflex and Fabric networks stay independently governable.

5. Network and Connectivity Architecture



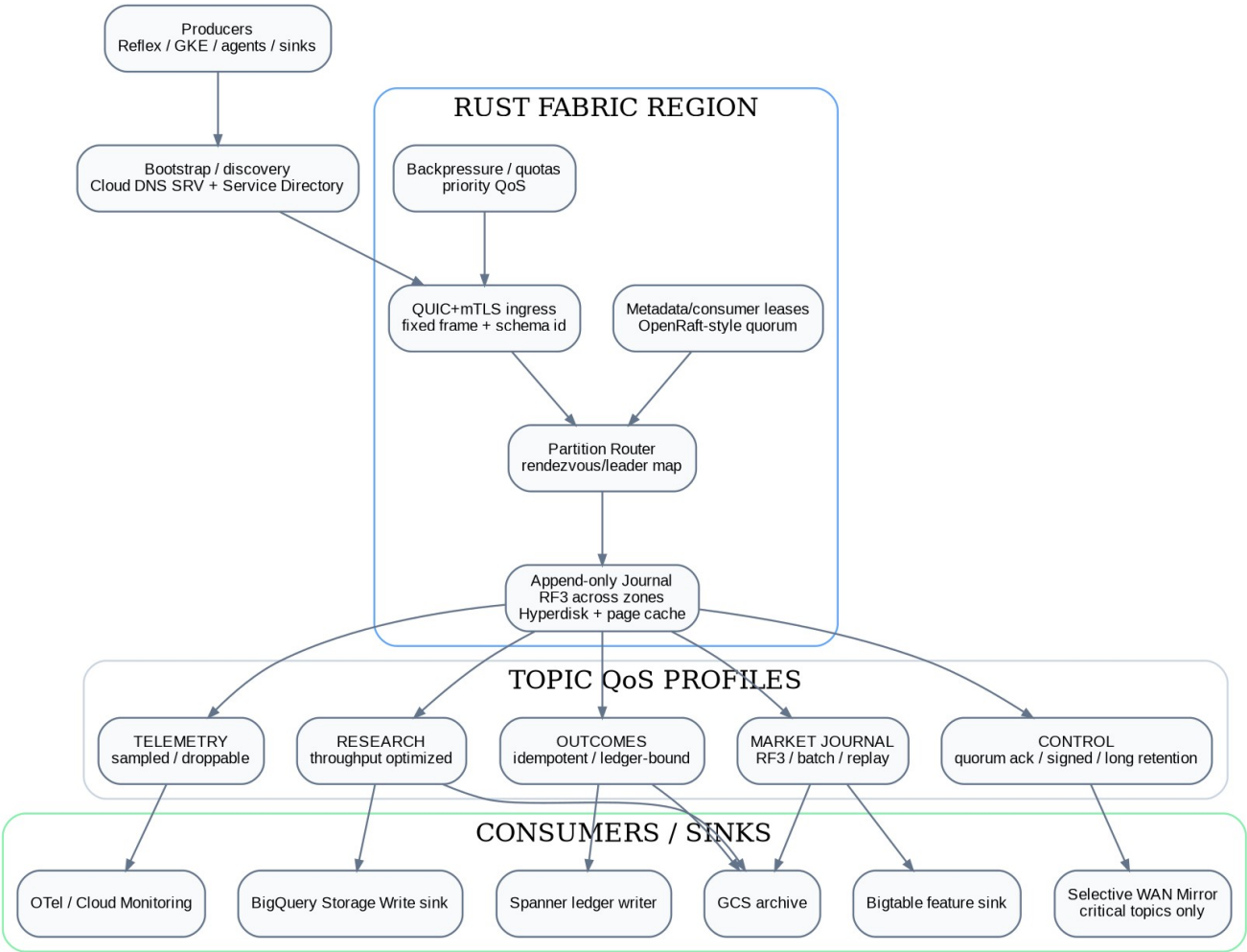
Path	GCP services / protocol	Design
Users -> Portal/API	Cloud DNS -> Global External Application Load Balancer -> Cloud Armor -> Cloud CDN/Certificate Manager -> Cloud Run or GKE Gateway	Anycast edge, WAF/DDoS controls, TLS, cache static assets; no direct access to internal planes.
Regional services	GKE Gateway API / regional internal Application LB + Cloud Service Mesh	mTLS, policy, retries/timeouts only for warm services; never inserted in Reflex path.
VPC-to-VPC	Network Connectivity Center VPC spokes with route filters	Central route orchestration without flattening all networks.
Latency mesh	Dedicated NCC mesh hub for Reflex/Fabric narrow prefixes	Only peer endpoints/ports are exported; direct cross-region Reflex and Fabric mirror traffic.
Service/data hub	NCC star/hybrid-inspection hub	GKE/data/AI/management connectivity with stronger centralized inspection.
Market/custody connectivity	Direct venue internet path where necessary; Dedicated/Partner Cloud Interconnect + Cloud Router for private providers; HA VPN as fallback	Avoid generic proxy path for latency-sensitive venue traffic; private hybrid connectivity when counterparties support it.
Google APIs	Private Google Access / Private Service Connect	Keep Spanner, GCS, Bigtable, Artifact Registry and other Google API traffic private where supported.
Controlled Internet egress	Cloud NAT or Secure Web Proxy for crawlers/agents/builders	Static egress identities, audit and domain controls; Reflex venue egress is separately governed.
Admin access	IAP / workforce identity / PAM	No public SSH/RDP; temporary privileged access with audit.

6. Regional Reflex Cell - Speed-Critical Execution



- **Compute:** C4D/C4-class Compute Engine sized by venue/instrument shard. Use gVNIC/Tier-1 networking where benchmarked beneficial; capacity reservations for production.
- **Host model:** run the Rust Reflex binary directly under systemd on a hardened minimal image. Avoid Kubernetes/service-mesh jitter in the decision loop.
- **OS tuning:** isolated CPU sets, CPU affinity, huge pages, mlockall, tuned IRQ/RPS/XPS, bounded background services, local scratch SSD and deterministic allocator/profile choices. Validate every tuning under replay.
- **State:** order books, features, compact reflex models, active risk envelope, session/inventory cache, instrument metadata and sequence state in memory; journal asynchronously to Fabric.
- **Venue sessions:** direct FIX/WebSocket/native adapters, per-venue connection pools and pre-allocated buffers. No generic HTTP service between Reflex and venue.
- **Failover:** fenced warm standby in another zone. Activation requires liveness failure + fencing epoch/venue-session ownership checks; never run two active owners accidentally.
- **Package manager:** downloads signed model/policy bundles outside the hot loop, verifies manifest/KMS signature, stages, health-checks, then atomically activates at an epoch boundary; retains instant local rollback.
- **Degraded mode:** loss of Fabric/GKE/global AI does not immediately halt. Reflex continues under last valid envelope, progressively narrows exposure according to local policy, then exits/halts if freshness limits expire.

7. Algorik Rust Event & Control Fabric



The Fabric is an Algorik product subsystem written in Rust. It is not a clone of Kafka feature-for-feature. It implements only the semantics the platform needs, with explicit priority classes and failure behavior.

Rust component	Responsibility	Implementation direction
fabricd	Broker: ingress, partition routing, journal replication, consumer fetch	Tokio async runtime; QUIC/mTLS; bounded memory pools; direct/buffered I/O chosen by benchmark.
fabric-meta	Cluster membership, partition maps, epochs, consumer leases, quotas	Small replicated metadata quorum using a mature Raft implementation; metadata separated from data log.
fabric-client	Producer/consumer SDK	Rust-first zero-copy buffers; idempotent producer sequence; async batching; explicit ack profile.
fabric-schema	Schema registry + compatibility checks	Protobuf/prost wire schema with immutable schema IDs and compatibility gates; generated Rust types.
fabric-mirror	Selective inter-region replication	Mirror only configured topics/partitions; independent lag budget; never blocks local commit.
fabric-archive	Segment compaction/archive to GCS	Closed immutable segments + manifest/checksum; replay can hydrate from GCS.

Rust component	Responsibility	Implementation direction
fabric-sink-*	Bigtable, BigQuery, Spanner, GCS consumers	Isolate destination backpressure from brokers; restart/replay by offset.
fabric-admin	Internal control API/CLI	Create topics, quotas, retention, partition moves, drain, diagnostics; mTLS and IAM-bound access.
fabric-observe	Metrics/traces/health	Native OTel metrics; fabric observability does not rely on its own event topics for liveness.

7.1 Wire, ordering and delivery semantics

- **Transport:** QUIC over gVNIC for producer/consumer and inter-broker sessions; TLS 1.3/mTLS by default. Benchmark TCP for specific long-lived bulk mirror flows rather than assuming one protocol wins everywhere.
- **Frame header:** magic/version, message type, schema ID/version, flags, partition, producer ID/epoch, sequence, event ID, source timestamp, logical timestamp, trace ID, payload length and CRC32C.
- **Partitioning:** explicit ordering key; rendezvous/consistent assignment to partition leaders; leaders replicate to followers in other zones.
- **Durability:** RF3 across zones for durable profiles. A quorum acknowledgment means the record survived on a majority of replicas. Cross-region mirroring is normally asynchronous.
- **Delivery:** at-least-once to consumers. Financial consumers are idempotent; producer epoch/sequence prevents stale duplicate producers; per-partition offsets provide replay.
- **Atomicity:** atomic append batches within one partition. Cross-partition business transactions are represented as sagas/commands, not hidden broker transactions.
- **Backpressure:** credit/window flow control, per-producer quotas, bounded queues and topic priority. Low-priority telemetry is sampled/dropped before control/outcome traffic is starved.
- **Schema evolution:** backward/forward compatibility policy at CI and runtime registration; rejected incompatible schemas never reach production topics.

7.2 Fabric topic classes

QoS class	Examples	Ack / replication	Retention / overload behavior
P0 Critical Control	risk envelope, model activation, kill/fence, capital grant	RF3 quorum, signed payload, producer fencing	Long retention + GCS archive; never intentionally dropped.
P1 Financial Outcomes	fills, cancels, position deltas, settlement events	RF3 quorum, idempotent IDs	Long hot retention + immutable archive; ledger sink retries indefinitely.
P2 Market Journal	ticks/books/features/decision traces	RF3 regional quorum; batch producer	Hours/days hot by instrument tier; archive to GCS; throttles mirror before local ingest.
P3 Intelligence/Research	evidence deltas, episodes, training candidates	RF2/3 configurable; throughput batching	Replayable; backlog allowed; scale consumers.
P4 Telemetry	metrics-derived events, debug traces	Best effort or RF2	Sample/drop oldest under pressure; direct OTel remains primary health path.

7.3 Fabric placement on GCP

- **Brokers:** five C4D/C4 VM instances per production region initially, spread across three zones. Three is minimum; five gives maintenance headroom and reduces correlated quorum pressure.
- **Journal disks:** persistent Hyperdisk sized for sustained append/read throughput, with local SSD/page cache for transient acceleration. Durability derives from replicated journal, not one local device.
- **Discovery:** Cloud DNS SRV records and optionally Service Directory provide bootstrap endpoints; clients obtain the live broker/partition map from Fabric metadata.
- **IP identity:** stable internal addresses, no public broker endpoints. Cross-region mirror prefixes are explicitly exported through the latency NCC hub.

- **Certificates:** Certificate Authority Service issues short-lived workload certificates; Cloud KMS/HSM protects signing roots/policy keys; broker identities map to GCP service accounts/IAM policy.
- **Archival:** closed segments are content-addressed and written to regional GCS landing buckets, then lifecycle/copied to durable research/archive locations according to residency policy.
- **Scaling:** add brokers and rebalance partitions. Scale partitions ahead of peak; do not auto-scale brokers reactively in the same way as stateless pods.
- **Upgrade:** one broker at a time: drain leadership, verify replica health, replace immutable image/binary, rejoin, then advance. Never upgrade enough replicas simultaneously to lose quorum.

8. Regional GKE Service Plane

Workload	Placement	Reason / scaling
Asset / Portfolio Brain	Regional GKE Standard	Stateful logic backed by Spanner/Bigtable; HPA on CPU/custom metrics.
Capital / Treasury Brain	Regional GKE Standard	Warm capital recommendations/grants; writes authoritative state through controlled services.
Risk Brain	Regional GKE Standard	Scenario/stress and envelope generation; hard pre-trade veto remains local Reflex.
Evidence / Provenance	GKE + Spanner Graph/BigQuery	Continuous source scoring and contradiction handling.
Fabric consumers/sinks	Dedicated node pools	Scale from consumer lag; destination-specific failure isolation.
Replay / Simulation controllers	GKE Jobs + batch compute	Launch replay shards, compare live vs counterfactual models.
Symbolic reasoners / solvers	Compute-optimized GKE node pools	Z3/OR-Tools/custom Rust graph/symbolic services; no hot-path dependency.
Crawler/scout workers	Isolated GKE node pools / Cloud Run jobs where appropriate	Separate egress, quotas and security posture from trading.
Internal APIs	GKE Gateway / internal Application Load Balancer	Regional service entry with mTLS/service policy.
Burst stateless endpoints	Cloud Run	Use for webhooks/control APIs whose cold-start/latency profile is acceptable; not for venue execution.

- **Cluster type:** GKE Standard regional clusters; regional control plane survives a single-zone loss and node pools span multiple zones.
- **Node pools:** separate general, compute, memory, GPU and batch/Spot pools. Taints/tolerations and PriorityClasses prevent research jobs from starving control workloads.
- **GitOps split:** Config Sync owns cluster baseline/policy/CRDs; Argo CD owns applications; Kargo owns environment promotion; Argo Rollouts owns progressive rollout inside a target cluster.
- **Secrets:** Workload Identity plus Secret Manager/CSI; do not persist long-lived cloud credentials in Kubernetes Secrets.
- **Service mesh:** Cloud Service Mesh only for warm GKE services; strict timeouts/retries/circuit breaking and mTLS. Do not route Fabric or Reflex traffic through mesh sidecars.

9. World, Tick and Replay Data Platform

9.1 Pass-Through World Sensing Pipeline

EXTERNAL SOURCE -> SECURE WEB/API EGRESS -> TRANSIENT SCOUT WORKER -> PARSE/EXTRACT -> EVIDENCE/PROVENANCE -> KNOWLEDGE DELTA

| -> SOURCE MANIFEST / HASH / CURSOR
 | -> SPANNER GRAPH / BIGTABLE / VECTOR+MEMORY
 ` -> RAW PAYLOAD DISCARDED

There is intentionally no general-purpose raw-world-data lake. The durable plane stores only Algorik's compact interpretation of the world and the metadata required to revisit the source. This lowers cost, reduces licensing/privacy exposure and keeps the cognitive platform focused on knowledge rather than copied content.

9.2 Durable Knowledge Tiers

- Tier K0 - live ambient state: in-memory/GKE caches and Bigtable for seconds-to-hours of compact derived state.
- Tier K1 - structured knowledge: Spanner Graph for durable entity, relationship, causal, belief and provenance-reference state.
- Tier K2 - cheap knowledge packs: GCS Standard/Nearline/Coldline for compressed protobuf/Parquet summaries, embeddings, model/world-state snapshots and episodic memory packs.
- Tier K3 - analytical knowledge: BigQuery tables contain derived facts/features/outcomes and research datasets, not copied external documents.
- Tier T - retained market/internal history: separate GCS/Tick Lake retention for tick/order-book and Algorik-owned execution/financial data under entitlements and policy.



Store/service	Owns	Connection from Fabric / reason
Cloud Storage regional landing	Retained tick/internal replay segments and compact derived knowledge packs; never a raw external-world corpus	fabric-archive writes closed segments via private Google APIs. Cheap durable replay foundation.
Cloud Storage durable research/archive	Curated internal/tick datasets, compact knowledge snapshots, model weights, manifests/hashes and legally required internal evidence	Lifecycle from landing; Bucket Lock/versioning where retention must be immutable.
Bigtable multi-cluster	Recent market features, replay indexes, high-rate entity/time series	fabric-sink-bigtable consumes partitions; multi-cluster routing for read resilience, app profiles chosen by consistency need.
BigQuery	Historical research, backtests, P&L attribution, data quality, feature studies	fabric-sink-bigquery / batch loads from GCS; isolated from hot path.
Knowledge Catalog	Data products, lineage, quality, business/technical context	Registers GCS/BQ/Spanner/AI lineage and evidence contracts; grounds agents in governed context.
Spanner Graph	World/causal/action-affordance graph and connected state	Evidence/world-model workers write normalized entities/edges; graph queries serve cognition, not hot execution.
AlloyDB	Operational relational metadata/workflow state that is not financial truth	Admin/research workflow state, agent jobs, configuration history where PostgreSQL semantics are useful.
Memorystore Redis Cluster	Ephemeral caches, rate limiting, warm coordination hints	Never authoritative; loss causes cache rebuild/degraded latency, not lost financial state.
Dataflow / Dataproc Serverless	Large batch reprocessing, enrichment, historical transforms	Reads durable GCS/BQ sources; not the primary live Fabric consumer path.

10. Ledger, Capital and Financial State

- **Spanner Enterprise Plus:** multi-region authoritative ledger/accounting database with external consistency; separate databases/instances for ledger vs world graph to avoid noisy-neighbor coupling.
- **Write model:** regional ledger-writer consumes P1 outcome partitions, opens a Spanner transaction keyed by event_id, updates double-entry postings/positions/cash, records processed event ID, and commits. Duplicate delivery becomes a no-op.
- **Leader placement:** choose Spanner configuration/leader region based on write origin and legal residency. Ledger is asynchronous to order placement, so global consistency does not slow the hot path.

- **Autoscaling:** use Spanner managed autoscaling with a deliberately high minimum sized for peak financial close/reconciliation periods; autoscaling is capacity management, not a substitute for hotspot-safe keys.
- **Change propagation:** Spanner change streams/Dataflow may feed BigQuery/reporting and derived caches; that is downstream analytics, not the system of record.
- **Reconciliation:** continuously compare venue/custody truth with ledger; discrepancies create a P0/P1 Fabric event and can narrow/stop local execution under deterministic policy.
- **Backups/audit:** scheduled Spanner backups/PITR where configured plus immutable ledger export/audit manifests to locked GCS for independent reconstruction evidence.

11. Cognitive, Model and Quantum Compute Plane

Capability	GCP placement	Interaction
Foundation/generalist & ambient agents	Gemini Enterprise Agent Platform / managed Agent runtime where appropriate; GKE for custom persistent agents	Consume Knowledge Catalog/Spanner Graph/BQ evidence; publish research tasks/results through Fabric.
Custom model training	Agent Platform / Vertex custom training and pipelines on GPU/TPU/CPU	Training datasets from GCS/BQ/Bigtable snapshots; output versioned model artifacts.
Model registry/evaluation	Model Registry / ML metadata	Model versions, evaluation, lineage, aliases and promotion evidence.
World-model federation	Spanner Graph + GKE/agent workers	Multiple competing models/graphs maintained and scored; disagreement becomes research signal.
Symbolic/neuro-symbolic reasoning	GKE compute pools	Z3/OR-Tools/custom Rust/Python reasoners; machine-checkable proofs/constraints where possible.
Simulation/counterfactual	GKE Jobs + batch compute; BigQuery/GCS history	Market replay, no-action branches, intervention simulation and strategy validation.
Quantum	Controlled egress gateway to IBM Quantum	Only async eligible optimization/search/QML research. Classical benchmark and validation required before any artifact can promote.
Reflex distillation	Training pipeline -> OCI/model package	Large model outputs distilled/compiled into bounded compact Reflex models and signed policy bundles.

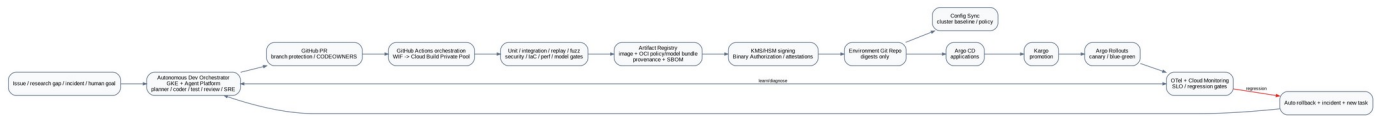
Model-package rule

A Reflex Node never calls a general model endpoint to decide a live order. It executes a locally loaded, tested, signed model/policy package with deterministic risk. General intelligence improves what gets packaged next.

12. Edge, Portal, API and Causal Action Plane

- **Public edge:** Cloud DNS + global external Application Load Balancer + Cloud Armor + Cloud CDN + Certificate Manager. Static content cached; dynamic BFF hosted in Cloud Run or GKE behind serverless/GKE backends.
- **Customer identity:** Identity Platform for customer/application identities; strong MFA/passkeys where supported. Internal workforce/admin paths use Cloud Identity/IAM/IAP/PAM.
- **API boundary:** public API exposes portfolio/ledger-derived views and authorized commands, never direct Spanner tables, Fabric broker ports, Reflex nodes or custody keys.
- **Causal action adapters:** separate GKE services for publication, outreach, product/pricing, market creation and operational tools. Each has action identity, jurisdiction/channel policy, rate limits, immutable audit and rollback/stop controls.
- **Crawler/scout isolation:** dedicated project/VPC/node pools and egress identity; untrusted web content cannot directly invoke high-authority tools or production secrets.

13. Autonomous Development and AI Engineering



Autonomous development is a controlled production capability. Agents can turn goals/incidents/research gaps into code and infrastructure changes, but they do not bypass Git, test, attestation or deployment policy.

Agent role	Allowed actions	Hard boundary
Planner	Read architecture/repo/issues/telemetry; decompose work; create plan	Cannot merge or deploy.
Coder	Create branch, edit Rust/Terraform/Kubernetes/model code, add tests	Sandboxed credentials; no prod secrets.
Test/Replay Agent	Run unit/integration/property/fuzz/replay/perf tests; compare baselines	Cannot waive failing risk/perf gates.
Security Agent	SAST/SCA/secrets/IaC/policy review; SBOM and vuln triage	Findings severity policy controls promotion.
Architecture Agent	Check dependency rules, hot-path violations, state ownership, ADRs	Cannot redefine non-negotiables silently.
Reviewer Agent	Independent review/diff risk summary; request changes	Separate model/context from primary coder for independence.
Release Agent	Update environment repo by immutable digest after gates	No direct kubect/SSH to prod.
SRE Agent	Observe rollout/SLOs, diagnose, trigger bounded rollback/runbook	Auto-remediation limited to signed runbooks and approved actions.
Learning Agent	Analyze incidents/reviews/benchmarks and create future tasks	May propose capability changes; promotion still gated.

- **Agent runtime:** control/orchestration on management GKE and/or managed Agent runtime; execution workers are ephemeral GKE Jobs/Cloud Build workers with scoped service accounts.
- **Sandboxing:** ephemeral namespaces/projects, NetworkPolicy/egress policy, quota, read-only production telemetry access by default, synthetic/test data when possible.
- **Model gateway:** agents may use Gemini/Claude/OpenAI/other approved models through one policy-enforcing gateway; credentials remain in Secret Manager and requests are audited.
- **Architecture memory:** ADRs, contracts, service catalog, SLOs, runbooks and blueprint are indexed for agents. Agents must cite the rule they are changing when proposing an exception.
- **No invisible changes:** an autonomous repair that changes code/config becomes a normal Git commit/PR or a pre-approved runtime action whose event is durably audited.

14. CI/CD, Supply Chain and GitOps

14.1 Repository model

Repository	Owns
algorik-app	Rust services, Reflex, Fabric, APIs, shared crates, tests.
algorik-models	Training code, evaluation, feature definitions, distillation, model manifests.
algorik-infra	Terraform modules/stacks for org/projects/network/GKE/Spanner/data/build/security.
algorik-platform-config	Config Sync cluster baseline, policies, CRDs, observability agents, GitOps controllers.

Repository	Owns
algorik-env	Environment desired state: application/model/policy digests and per-region rollout declarations.
algorik-schemas	Fabric event schemas, compatibility rules, generated Rust clients/types, data contracts.
algorik-runbooks	SRE runbooks, game-day definitions, remediation policy, architecture acceptance tests.

14.2 Source-to-production flow

1. GitHub branch/PR is created by a human or authorized agent. Branch protections require tests and independent review based on risk class.
2. GitHub Actions orchestrates workflow and exchanges OIDC for short-lived GCP credentials through Workload Identity Federation; no stored service-account keys.
3. Cloud Build private pool performs reproducible builds inside private networking. Rust builds use locked dependencies, cargo-deny/audit, clippy, unit/property/fuzz tests, replay and benchmark gates.
4. Terraform plan, Kubernetes policy checks, secret scan, SAST/SCA, container scan and architecture rule tests run before artifacts are accepted.
5. Artifact Registry stores images, Rust packages and OCI deployment bundles by immutable digest; SBOM/provenance/scan results are attached.
6. Cloud KMS/HSM signs release/model/policy manifests; Binary Authorization and admission policy require trusted attestations for GKE workloads.
7. Release Agent updates algorik-env with exact digests. Config Sync reconciles platform baseline; Argo CD reconciles application desired state.
8. Kargo promotes the same immutable artifact through dev -> integration -> replay -> paper -> staging -> prod regions. Rebuild-per-environment is prohibited.
9. Argo Rollouts performs canary/blue-green for GKE services. Reflex/Fabric use component-specific drain/stage/epoch activation rather than generic pod canaries.
10. Cloud Monitoring/Prometheus/replay comparators enforce rollout SLOs. Regression automatically rolls back and creates an incident/task for the autonomous dev loop.

14.3 Component-specific release mechanics

Component	Deployment primitive	Safe rollout
GKE services	Argo CD + Kargo + Argo Rollouts	Canary by traffic/replicas; SLO gate; instant previous digest rollback.
Reflex binary	Immutable VM image/binary + package manager	Standby first -> replay shadow -> drain/fence -> activate epoch -> compare -> expand.
Fabric broker	Immutable VM image / controlled instance replacement	Drain leaders/partitions -> verify RF -> replace one broker -> rejoin -> repeat.
Models/policy	Signed OCI manifest + GCS weights	Shadow score -> paper/replay -> stage locally -> atomic epoch activation -> rollback retained.
Terraform	GitHub Actions / Cloud Build plan + gated apply	Folder/project/network/database changes have explicit change windows and plan artifact.
Schemas	Schema registry + CI compatibility gate	Additive compatible first; dual-read/write migrations; retire only after consumer inventory proves safe.

15. Security and Trust Architecture

Control	Use
Cloud Identity / IAM	Human/workload identity; least privilege; group-based access.
Privileged Access Manager	Time-bounded privileged elevation and approvals for sensitive operations.
Workload Identity Federation	GitHub/external CI obtains short-lived GCP access without service-account keys.
Workload Identity for GKE	Pods map to GCP service identities; no node-wide credentials.
Secret Manager	Venue/API credentials and non-key secret material; rotation/versioning/audit.
Cloud KMS / Cloud HSM	CMEK, package signing, high-assurance key material and attestation roots.
Certificate Authority Service	mTLS certificates for Fabric/VM and internal workload identities outside service mesh.
VPC Service Controls	Perimeters around ledger/data/AI projects to reduce exfiltration paths.
Cloud Armor	Internet-facing WAF/DDoS/rate controls.
Cloud NGFW / hierarchical firewall policies	Organization/folder network policy, egress segmentation and narrow lateral paths.
Security Command Center	Posture, findings, threat/vulnerability aggregation and response integration.
Artifact Analysis + Binary Authorization	Supply-chain scanning and deploy-time enforcement.
Sensitive Data Protection	Discovery/classification/redaction policy for PII/sensitive data in lake/analytics.
Cloud Audit Logs + Cloud Asset Inventory	Administrative/data access evidence and inventory/change visibility.
IAP	Internal/admin web access without public management endpoints.

Key separation

Custody/transfer keys, deployment-signing keys, Fabric identity keys and user-auth keys are separate trust domains. An engineering agent compromise must not yield authority to move capital, and a trading-service compromise must not yield deployment-signing authority.

16. Observability, SRE and AIOps

- **Instrumentation:** OpenTelemetry SDK/collector everywhere. Reflex/Fabric export asynchronously to regional collectors; no logging call may block order processing.
- **Metrics:** Managed Service for Prometheus + Cloud Monitoring. Golden signals plus domain metrics: decision p99, venue ack, Fabric quorum ack, partition lag, mirror lag, ledger commit, reconciliation breaks, model freshness and package activation.
- **Logs:** Cloud Logging for application/control logs with strict sampling on high-rate paths; raw tick/decision history belongs in Fabric/GCS, not verbose log ingestion.
- **Traces:** Cloud Trace for warm/API workflows; sampled correlation IDs bridge into Reflex/Fabric event metadata without forcing distributed-trace network calls.
- **Network:** VPC Flow Logs where appropriate, Connectivity Tests/Network Intelligence tooling, load-balancer telemetry and Interconnect/VPN metrics.
- **SLOs:** per plane and dependency, with error budgets that drive release policy. Fabric backlog and ledger reconciliation are product SLOs, not only infrastructure metrics.
- **AIOps:** SRE agent reads telemetry, compares runbooks and recent diffs, opens incidents/PRs, scales permitted services and can execute pre-approved rollback/restart/drain actions with audit.

Component	Primary SLO examples
Reflex	market-event-to-decision p99/p99.9; venue round-trip; stale-book age; local risk-gate latency; dropped feed messages.
Fabric	publish quorum p99; broker availability; partition under-replication; consumer lag; mirror lag; archive lag; schema reject rate.
GKE	request p99/error; saturation; pod disruption; dependency circuit opens.
Ledger	Spanner commit p99; outcome-to-ledger lag; reconciliation mismatch age; transaction abort rate.
Data	GCS retained-archive delay; source re-fetch health; Bigtable p99; knowledge-delta freshness; BQ ingestion freshness; lineage completeness.
AI	model package freshness; calibration/drift; training/evaluation queue; agent tool failures; quantum/classical benchmark delta.
Delivery	lead time; rollback rate; build reproducibility; attestation coverage; config drift; autonomous PR acceptance/rework rate.

17. Availability, Resilience and Disaster Recovery

Failure	Expected behavior	Recovery / target
Reflex process/VM	Standby remains fenced until failover criteria; active venue connection lost only for that shard.	Restart/standby promotion after liveness + fencing + venue-session checks. Measure target per venue; do not promise zero-gap failover.
One GCP zone	Fabric RF3 and regional GKE retain quorum/control plane; Reflex can promote standby if primary zone lost.	Automated managed-service recovery; verify partition replicas before maintenance.
Fabric broker	Partition leader moves to follower; producers refresh map.	Quorum-acked records preserved; replace broker and rebalance.
Entire Fabric region	Reflex can continue under cached policy; GKE async workflows lose local bus; critical mirrored topics available elsewhere.	Bring region back from surviving replicas/GCS archives; healthy regions continue.
Regional GKE	Reflex continues; Fabric continues journaling; global/other-region services take research/control workload.	Rebuild cluster from Terraform + Config Sync + Argo; use another region for eligible APIs.
Global cognition/AI	No new models/research; local trading follows freshness/degradation envelope.	Retry/fail to alternate AI region; no hot-path impact.
Spanner leader region	Spanner moves leadership according to instance configuration; ledger may see latency change.	Managed service HA; monitor outcome-to-ledger backlog.
Bigtable cluster	Multi-cluster routed reads fail to another cluster; single-cluster transactional workloads require explicit failover.	App profiles define behavior; replay from Fabric/GCS if necessary.
GCS research region	Use replicated/dual-region locations according to residency policy; regional landing remains source segment until archived.	Rehydrate derived stores from immutable objects.
GitOps management plane	Running workloads do not stop; desired-state reconciliation pauses.	Recreate from Git; failover management cluster if required.
Artifact Registry region	Existing nodes keep local images/packages; new rollout pauses.	Replicate/publish release artifacts to designated secondary repository before prod promotion.

Failure	Expected behavior	Recovery / target
Autonomous dev plane	No autonomous changes; production keeps running.	Human/manual pipeline remains available; agents are convenience, not availability dependency.

18. Scaling and Performance Model

Subsystem	Scale unit	Scaling rule
Reflex	venue/instrument shard / VM	Scale intentionally by sharding sessions/instruments and pre-provisioning; no reactive autoscale during a market burst.
Reflex Mesh	peer connections / opportunity domains	Exchange compressed state only; bound fan-out and message sizes; do not broadcast raw tick streams globally.
Fabric	brokers + partitions	Add brokers, move leaders/followers, increase partitions by domain; keep headroom for zone loss and maintenance.
GKE services	pods/node pools	HPA/VPA and custom lag/queue metrics; Cluster Autoscaler for warm/batch workloads.
Bigtable	nodes/clusters	Managed autoscaling with hotspot-safe row keys and workload-specific app profiles.
Spanner	processing units/nodes	Managed autoscaling with high floor; schema/key design eliminates hotspots before scaling.
BigQuery	slots/reservations	Separate research, backtest and BI reservations/workload management so one job cannot monopolize compute.
GCS	object/segment parallelism	Partition by source/date/instrument and write immutable reasonably sized segments; compact tiny objects asynchronously.
AI training	GPU/TPU workers/jobs	Queue and quota-aware scheduling; Spot where interruption is safe; fixed reserved capacity for critical retraining if justified.
Autonomous dev	agent workers/build pool	Ephemeral workers and build private pools scale independently; concurrency quotas protect source APIs and budgets.

19. Complete GCP Service Catalog for This Platform

GCP service	Role in Algorik
Cloud Resource Manager / Folders / Projects	Resource hierarchy and blast-radius isolation.
Cloud Identity	Workforce identity.
IAM	Authorization for users/workloads.
Privileged Access Manager	Just-in-time privileged access.
Organization Policy	Non-bypassable org guardrails.
Cloud Asset Inventory	Asset/change inventory.
Security Command Center	Security posture/findings.
Cloud Audit Logs	Administrative/data-access evidence.

GCP service	Role in Algorik
VPC	Isolated Reflex/Fabric/Service/Data/Engineering networks.
Network Connectivity Center	VPC/hybrid hub-spoke connectivity with route filters.
Cloud Router	Dynamic BGP for Interconnect/VPN.
Dedicated/Partner Cloud Interconnect	Private provider/colo/custody connectivity where available.
HA VPN	Fallback hybrid connectivity.
Cloud NAT	Controlled internet egress for non-latency-critical workloads.
Secure Web Proxy	Policy/audit for agent/crawler web egress where suitable.
Private Service Connect	Private access to producers/services.
Private Google Access	Private Google API access.
Cloud DNS	Public/private zones and Fabric bootstrap SRV.
Service Directory	Optional internal endpoint discovery.
Cloud Load Balancing	Global edge and regional internal ingress.
Cloud Armor	WAF/DDoS/rate control.
Cloud CDN	Public/static acceleration.
Certificate Manager	Public TLS certificates.
IAP	Protected admin interfaces.
Cloud NGFW / hierarchical firewall policies	Network segmentation and organization policy.
Compute Engine C4D/C4-class	Reflex and Fabric dedicated VMs.
Local SSD	Reflex scratch/cache and optional Fabric transient acceleration.
Hyperdisk	Persistent Fabric broker journals / VM disks.
Reservations / commitments	Guarantee production hot capacity and control cost.
Shielded VM	Boot integrity for VM-based critical services.
GKE Standard regional clusters	Warm service, solver, agents and sink compute.
Cloud Service Mesh	mTLS/traffic policy for warm GKE services only.
Cloud Run	Bursty stateless web/control jobs where latency profile allows.
Cloud Storage	Tick/internal replay lake, Fabric segment archive, model weights, compact knowledge packs and internal evidence. External world payloads are not retained.
Bigtable	High-rate hot time-series/features/replay indexes.
BigQuery	Research, backtesting, attribution, analytics.
Knowledge Catalog	Governance, lineage, quality, context and data products.
Dataflow	Large batch/reprocessing and Spanner-change-stream pipelines; not primary live bus.
Dataproc Serverless	Optional Spark batch/large historical jobs.
Memorystore Redis Cluster	Ephemeral cache/rate/coordination hints.
AlloyDB	Operational relational metadata/workflows.
Spanner Enterprise Plus	Ledger/capital/accounting global consistency and multi-region HA.

GCP service	Role in Algorik
Spanner Graph	World/causal/affordance knowledge graph in separate workload boundary.
Gemini Enterprise Agent Platform / Vertex AI APIs	Training, managed agents, evaluation and model lifecycle.
Model Registry / ML Metadata	Version/evaluation/lineage for models.
Cloud Build private pools	Private reproducible CI/build/test workers.
Artifact Registry	Images, packages and OCI release/model manifests.
Artifact Analysis	Vulnerability metadata/scanning.
Binary Authorization	Deploy-time attestation enforcement.
Workload Identity Federation	Keyless GitHub/external CI access.
Secret Manager	Venue/API/service secrets.
Cloud KMS	CMEK/signing.
Cloud HSM	Higher-assurance signing/key material where required.
Certificate Authority Service	Private mTLS identity for Fabric/VMs.
VPC Service Controls	Data/AI/ledger exfiltration perimeter.
Sensitive Data Protection	Sensitive-data discovery/classification/redaction.
Cloud Monitoring	Metrics/SLOs/alerts.
Managed Service for Prometheus	Prometheus metrics at scale.
Cloud Logging	Control/application logs.
Cloud Trace	Warm/API trace analysis.
Network Intelligence Center / Connectivity Tests	Network diagnostics/topology/connectivity verification.
Billing export / Budgets	FinOps and spend guardrails.

19.1 Deliberately not used as core internal messaging

Service	Decision
Google Pub/Sub	Not used for Algorik internal pub/sub/control/outcome/tick journals. Native Rust Fabric defines those semantics.
Managed Service for Apache Kafka	Not used as core internal event backbone. Optional external compatibility bridge only if a partner/client requires Kafka protocol.
Eventarc	Not part of core internal eventing; may be used only for peripheral Google-service automation if it does not become an application dependency.
Cloud Tasks	Not an internal general queue. May be used by isolated customer-facing workflows only when its task semantics specifically fit.

20. Critical Service-to-Service Connection Matrix

From	To	Protocol	Mode	Reason
Venue feed	Reflex Node	FIX/WS/native TCP/UDP	Sync hot	Market data and orders; only true trading path.

From	To	Protocol	Mode	Reason
Reflex Node	Peer Reflex Node	Direct QUIC	Fast ephemeral	Cross-region opportunity/reservation coordination; no durable replay requirement.
Reflex Node	Regional Fabric	QUIC/mTLS	Async	Journal ticks/decisions/outcomes/control acknowledgments; failure cannot block order.
GKE control service	Regional Fabric	QUIC/mTLS	Async	Publish policy, research, capital/risk envelope events.
Regional Fabric	GKE consumers	QUIC/mTLS	Async pull/stream	Warm services consume ordered partitions and replay.
Fabric Archive	Cloud Storage	Google API private access	Async	Immutable segment archive/replay source.
Fabric Bigtable Sink	Bigtable	gRPC	Async	Recent features/time-series/replay indexes.
Fabric BigQuery Sink	BigQuery Storage Write API	gRPC	Async	Research/analytics stream; can lag without trading impact.
Ledger Writer	Spanner	gRPC	Async transactional	Idempotent authoritative financial commit.
Evidence/World workers	Spanner Graph	gRPC/SQL/GQL	Warm	Maintain world/causal/action-affordance graph.
Training pipelines	GCS/BQ/Bigtable/Spanner Graph	Private Google APIs	Batch/warm	Build training/evaluation datasets.
Training/Model Registry	Artifact Registry + GCS	Google APIs	Async	Publish deployment package manifest/weights.
Package Manager	Artifact Registry/GCS + KMS verification	HTTPS/gRPC private	Warm	Stage signed model/policy bundle before activation.
Agent/Symbolic workers	IBM Quantum gateway	HTTPS controlled egress	Async	Eligible quantum jobs only; no execution dependency.
Public client	Global ALB	HTTPS	Sync	User/API edge.
Global ALB	Cloud Run/GKE Gateway	HTTPS	Sync	Portal/BFF/API service.
GitHub Actions	GCP WIF	OIDC/STSes	CI	Short-lived deployment/build identity.
GitHub Actions	Cloud Build private pool	API	CI	Reproducible private build/test.
Cloud Build	Artifact Registry	HTTPS	CI	Push signed/provenance-bearing artifact.
Environment Git	Config Sync / Argo CD	Git/HTTPS	GitOps	Desired-state reconciliation.
OTel collectors	Cloud Monitoring/Prometheus/Logging/Trace	OTLP/Google APIs	Async	Telemetry; sampling/buffering protects production.

21. End-to-End Platform Flows

21.1 Tick to order and learning

1. Venue market data enters the regional Reflex adapter directly and is sequence-checked/normalized in memory.
2. Local book/features/reflex model evaluate the event; deterministic local risk gate approves/vetoes; order is sent directly to venue.
3. Tick, feature snapshot, decision, order and fill/cancel are asynchronously appended to Fabric P1/P2 topics. A Fabric outage cannot retroactively stall the order.
4. Fabric sinks archive market/decision segments to GCS, update recent features in Bigtable and commit financial outcomes idempotently to Spanner.
5. Replay/evaluation compares predicted fills/slippage/outcomes against actual results and emits episodes/model residuals.
6. Training pipelines use world + tick history to improve specialist/reflex models; candidates are evaluated, signed and promoted through GitOps/model gates.
7. Reflex package manager stages the new package and activates it atomically at the approved epoch; previous package remains rollback-ready.

21.2 World event to cognitive response

1. Scout/crawler/API connector captures an observation with source identity and provenance and publishes to regional Fabric plus immutable GCS evidence.
2. Evidence workers normalize entities, corroborate sources and update trust/confidence; world-model services update Spanner Graph.
3. Ambient models detect surprise/novelty or causal implications and publish research/opportunity tasks to Fabric.
4. Specialist agents/symbolic reasoners query Knowledge Catalog, Spanner Graph, BigQuery and simulations; quantum is invoked only for eligible asynchronous research.
5. Validated result becomes a strategy/model/risk/capital proposal. Promotion gates convert it into signed policy/model packages, never an unreviewed direct hot-path call.

21.3 Multi-region arbitrage

1. Each Reflex Cell maintains local venue state and computes local executable edges.
2. Compressed opportunity/reservation state is exchanged directly over Reflex Mesh, not through Fabric.
3. Coordinator/participants acquire bounded local reservations using opportunity epoch/fencing and execute legs under local deterministic risk.
4. Partial completion follows explicit hedge/unwind saga rules. No path is considered riskless when atomicity is impossible.
5. All intents/fills/hedges/outcomes are journaled to Fabric and ledgered centrally; replay evaluates missed/failed cycles and improves models.

21.4 Autonomous development change

1. Incident, backlog item or Intelligence Expansion gap creates a scoped engineering goal.
2. Planner/coder agents create a branch/PR and tests; independent test/security/architecture agents evaluate it in sandbox.
3. Cloud Build private pools build deterministic artifacts and run Rust replay/performance/fuzz plus security/IaC gates.
4. Artifact Registry stores immutable digest/provenance; KMS/HSM creates signing evidence/attestation.
5. Release agent changes only the environment Git repository. Kargo promotes the exact digest; Argo/rollout mechanics depend on component type.
6. SLO/replay gates observe production; regression triggers rollback and feeds evidence back to the agent loop.

22. Dependency Degradation Contracts

Dependency lost	Reflex behavior	Platform behavior
Fabric	Continue under local policy; buffer bounded critical events to local spool; narrow if spool/freshness policy exceeded.	GKE loses async bus; alert immediately; restore cluster and replay local/GCS segments.
GKE regional services	Continue local execution; no new warm policy/capital intelligence.	Fabric keeps journaling; other-region/global services may cover eligible functions.
Spanner	Continue only within cached capital/risk envelope until ledger-lag threshold; stricter transfer/position actions disabled.	Outcome partitions accumulate durably in Fabric; ledger writer replays idempotently when restored.
Bigtable/BigQuery	No direct hot impact.	Research/replay/features degrade; Fabric/GCS preserve source history.
AI/Agent Platform	No new cognitive/model output.	Existing packages continue; research backlog queues.
Cross-region network	Local trading continues; disable multi-region cycles requiring peers; Fabric mirrors lag.	Regional durability intact; catch up mirror when link returns.
Market data venue feed	Stop strategies dependent on stale feed; hedge/exit according to local playbook.	Incident and reconciliation events journaled.
GitHub/GitOps	Running system unchanged.	No new deployments; emergency runbook uses pre-approved break-glass process with later reconciliation to Git.

23. FinOps and Capacity Governance

- **Cost domains:** label/tag every project/resource by plane, region, environment, owner and workload; export billing to BigQuery and report cost per strategy/model/data source/region where attribution is possible.
- **Hot-path economics:** Reflex/Fabric capacity is intentionally overprovisioned for latency and failure headroom. Optimize with committed use/reservations only after benchmark sizing is stable.
- **Data retention:** tier tick/journal retention: high-value hot replay in Fabric/Bigtable, immutable historical segments in GCS, research aggregates in BigQuery. Do not retain every derived duplicate forever.
- **AI economics:** budget/concurrency quotas per agent/training family; Spot/preemptible training where safe; benchmark model benefit per dollar and latency.
- **Autonomous-agent budget:** agents get explicit daily/monthly model/build/tool budgets and can be throttled independently of production runtime.
- **Cost anomalies:** Budgets + Monitoring/BQ anomaly jobs create Fabric/incident tasks, but never auto-delete production state as a cost action.

24. Environment and Promotion Topology

Environment	Purpose	Data / connectivity
dev	Fast engineer/agent iteration	Synthetic/mock venues, reduced GKE/data, no production secrets.
integration	Cross-service contract and schema validation	Full Rust Fabric topology at small scale; deterministic integration feeds.
replay	Historical tick/world replay and performance regression	Read-only curated production-derived data with governance; no live order routes.
paper	Live market data, simulated orders	Real feed behavior; ledger isolated from real capital.
staging	Production-like scale/failure/game-day validation	Separate accounts/credentials; canary packages; no uncontrolled live capital.

Environment	Purpose	Data / connectivity
prod	Live bounded execution	Strict region/project isolation, reserved capacity, signed artifacts only.

Promotion invariant

Promote the same digest and schema/model version. Environment-specific behavior comes from signed configuration and authorized secrets, not rebuilding different code.

25. Resilience Testing and Game Days

- **Quarterly regional-loss game day:** remove one execution region from service/control networking; verify other regions continue, mirrors/recovery behave, and no cross-region arbitrage assumes missing peer.
- **Monthly zonal failure:** terminate Reflex primary or Fabric zone replicas; prove fencing/quorum/failover and bounded outcome loss.
- **Broker backpressure:** freeze a sink, fill consumer lag, verify priority QoS and no broker memory explosion.
- **Ledger outage:** deny Spanner writes, verify Fabric P1 retention and local capital freshness degradation, then idempotent catch-up.
- **Bad model/package:** promote intentionally degraded candidate in nonprod/staging; verify shadow/replay/SLO gate blocks or rolls back.
- **Bad schema:** attempt incompatible Fabric schema; CI/runtime registry must reject before production consumer breakage.
- **Agent compromise simulation:** give autonomous dev agent malicious instruction/input; verify secret, network, merge and prod-deploy boundaries prevent authority escalation.
- **Rebuild from zero:** periodically rebuild a regional GKE cluster and a nonprod Fabric cluster from Terraform/Git/artifacts, proving no undocumented console dependency.

26. Recommended Build Order

Phase	Deliverable / exit test
0 - Foundations	Org/folders/projects, IAM/PAM, Terraform modules, NCC/VPC/DNS/firewalls, KMS/CA, audit/SCC, GitHub WIF.
1 - Supply chain	Cloud Build private pools, Artifact Registry, provenance/SBOM, Binary Authorization, GitOps management cluster, Config Sync/Argo/Kargo/Rollouts.
2 - Rust Fabric MVP	Single region 3 brokers, schemas/client, RF3 journal, consumer offsets, GCS archive, OTel; chaos tests before application adoption.
3 - One Reflex Cell	One venue/asset shard, local risk/order flow, async Fabric journal, replay and paper trading.
4 - Financial truth	Spanner ledger writer/idempotency, reconciliation, capital/risk envelope contracts.
5 - Data plane	GCS retained tick/internal archive + compact knowledge packs, Bigtable, BigQuery, Knowledge Catalog, replay/feature sinks, data contracts.
6 - Regional service plane	GKE asset/capital/risk/evidence/API services; Fabric consumer scaling; Redis cache.
7 - Multi-region	Three regional Reflex/Fabric/GKE cells; NCC latency mesh; selective Fabric mirrors; direct Reflex Mesh; game-day failover.
8 - Cognitive/AI	Spanner Graph, model registry/training, ambient/symbolic agents, world+tick fusion, signed model package lifecycle.
9 - Autonomous development	Planner/coder/test/security/reviewer/release/SRE agent loop through normal CI/GitOps, with sandbox and budget controls.
10 - Quantum/agency expansion	IBM Quantum gateway, causal agency adapters, additional assets/markets, larger arbitrage cycles; only after classical baseline/reliability gates.

27. Architecture Acceptance Criteria

- COMPLETE TARGET: No synchronous call to Fabric, GKE, Spanner, Bigtable, Redis, BigQuery, AI or another region is required between a venue event and a local order.
- COMPLETE TARGET: Google Pub/Sub and Managed Kafka are absent from core internal messaging/dataflow; all internal async contracts use the Rust Fabric SDK/schema model.
- COMPLETE TARGET: A Fabric broker/zone can fail without losing quorum-acknowledged P0/P1/P2 records or stopping Reflex execution.
- COMPLETE TARGET: Every financial outcome is eventually committed exactly once in effect to Spanner through event-id idempotency, even when delivered multiple times.
- COMPLETE TARGET: Every model/policy executing in Reflex can be traced to source commit, data/evaluation manifest, signature, approval/promotion event and activation epoch.
- COMPLETE TARGET: Every GCP workload has a documented state owner, upstream/downstream protocol, scaling model and degraded mode.
- COMPLETE TARGET: Region loss does not globally halt healthy regions; cross-region cycles depending on failed peers are disabled explicitly.
- COMPLETE TARGET: Production GKE workloads are reconciled from Git and admitted only when supply-chain policy is satisfied.
- COMPLETE TARGET: Autonomous agents cannot obtain production capital/custody authority, cannot merge/deploy around policy, and cannot access long-lived cloud keys.
- COMPLETE TARGET: Replay can reconstruct market/decision/outcome history from Fabric/GCS sufficiently to test a strategy/model and explain a live incident.
- COMPLETE TARGET: Reconciliation can rebuild/verify positions and cash from authoritative ledger plus venue/custody statements without trusting caches.
- COMPLETE TARGET: Every critical component is exercised in scheduled game days and has observable RTO/RPO or a documented reason exact guarantees cannot be promised.

28. Explicit Anti-Patterns

- Putting the Rust Fabric in front of venue market data or order submission simply because it is fast. The hot path bypasses it.
- Rebuilding Kafka semantics blindly. Implement Algorik-required semantics and prove them with fault/property tests.
- Claiming global exactly-once delivery. Use at-least-once transport plus idempotent state transitions and fencing.
- Using one Spanner database for ledger, world graph, agent memory and every service table. Separate trust/performance domains.
- Using Redis as position/capital truth or as a required risk gate dependency.
- Using a single GKE cluster globally or a single giant flat VPC.
- Routing Reflex traffic through service mesh, proxy, Secure Web Proxy, Cloud NAT chains or general API gateways without benchmarked necessity.
- Allowing an AI agent to change production with kubectl/SSH or console clicks instead of GitOps or a pre-approved audited runbook.
- Allowing model promotion without replay/calibration/classical baseline and deterministic risk-envelope compatibility tests.
- Using mutable latest tags, unversioned schemas, hand-copied model files, or undocumented emergency config.
- Observability that blocks execution or emits raw tick rate directly into general-purpose log ingestion.

29. Current GCP Capability Validation Notes (2026)

The architecture relies on current GCP capabilities but keeps product semantics independent of them. Key validated platform facts:

- C4D supports very large CPU/memory configurations, local Titanium SSD, compact/spread placement policies and Tier-1 networking up to 200 Gbps; exact machine selection still requires venue-level benchmark testing.
- GKE regional clusters replicate the control plane across zones and are the recommended production availability mode for regional workloads.
- Spanner supports regional, dual-region and multi-region configurations; multi-region/dual-region Enterprise Plus configurations provide higher availability and strong/external consistency, at the cost of additional write latency.
- Spanner Graph provides graph/relational/search capabilities and is appropriate for a separate world/causal graph workload; do not share its noisy analytical load with the ledger database.
- Bigtable supports autoscaling and multi-cluster routing; routing/app-profile choice must match read/write consistency needs.

- Network Connectivity Center supports VPC and hybrid hub/spoke topologies and route filters, fitting the separated latency/service network model.
- Cloud Build private pools provide private-network build workers; Binary Authorization can enforce build/signing attestations at deployment.
- Config Sync provides GitOps synchronization and drift prevention across GKE fleets; in this design it owns baseline/policy while Argo CD/Kargo own applications/promotion.
- Knowledge Catalog (formerly Dataplex Universal Catalog) provides lineage, quality/governance and context features, including multi-region lineage capabilities.
- Google AI product branding is evolving toward Gemini Enterprise Agent Platform while Vertex AI APIs/features remain the underlying model/training ecosystem in current documentation; keep application interfaces behind Algorik adapters so branding/API evolution does not leak into product contracts.

30. Reference Documentation

- Google Compute Engine general-purpose C4/C4D machine family: <https://docs.cloud.google.com/compute/docs/general-purpose-machines>
- GKE regional clusters: <https://docs.cloud.google.com/kubernetes-engine/docs/concepts/regional-clusters>
- Spanner instance configurations: <https://docs.cloud.google.com/spanner/docs/instance-configurations>
- Spanner managed autoscaling: <https://docs.cloud.google.com/spanner/docs/managed-autoscaler>
- Spanner Graph overview: <https://docs.cloud.google.com/spanner/docs/graph/overview>
- Bigtable autoscaling/routing: <https://docs.cloud.google.com/bigtable/docs/autoscaling> and [/routing](https://docs.cloud.google.com/bigtable/docs/routing)
- Network Connectivity Center: <https://docs.cloud.google.com/network-connectivity/docs/network-connectivity-center>
- Cloud Build private pools: <https://docs.cloud.google.com/build/docs/private-pools/private-pools-overview>
- Binary Authorization with Cloud Build: <https://docs.cloud.google.com/binary-authorization/docs/cloud-build>
- GKE Config Sync overview: <https://docs.cloud.google.com/kubernetes-engine/config-sync/docs/overview>
- Knowledge Catalog: <https://docs.cloud.google.com/knowledge-catalog/docs>
- Artifact Registry: <https://docs.cloud.google.com/artifact-registry/docs/overview>

31. Final Platform Definition

Algorik on GCP is a deliberately asymmetric global system. Dedicated Reflex Cells own live venue decisions; direct Reflex Mesh links coordinate time-sensitive multi-region opportunities; a native Rust Event & Control Fabric provides the durable asynchronous nervous system; GKE runs the elastic regional brains and sinks; Spanner owns financial truth; GCS/Bigtable/BigQuery/Knowledge Catalog preserve and organize world/tick knowledge; Spanner Graph and the AI platform support multi-world, symbolic, ambient and causal intelligence; and GitOps plus autonomous engineering continuously evolves the system without granting agents a shortcut around controls.

Architecture mantra

Execute locally. Coordinate directly when time matters. Journal durably in Rust. Persist financial truth strongly. Store retained tick/internal history immutably; never warehouse raw external-world content. Reason globally. Package intelligence immutably. Promote through Git. Observe every plane. Fail independently. Rebuild from source. Let agents automate engineering - never authority.